

Median in Sorted Matrix

Median in a Row-Wise Sorted Matrix

Let's build this from **brute force** → **intuition** → **optimal binary search**.

1. What is the problem asking?

You are given a matrix where **every row is sorted**, but the columns are **not necessarily sorted**.

For example:

```
1 3 5
2 6 9
3 6 9
```

There are:

```
n × m = 3 × 3 = 9
```

elements.

If we put all elements into one array and sort:

```
[1, 2, 3, 3, 5, 6, 6, 9, 9]
```

The middle element is:

```
5
```

So the answer is 5.

Important observation

Since **n** and **m** are odd:

```
n × m
```

is also odd.

Therefore, there is always exactly **one middle element**.

Its position in sorted order is:

```
(n * m) / 2
```

using **0-based indexing**.

Or, equivalently, we need:

```
(n * m) / 2 + 1
```

elements to be $\leq$ **median**.

2. First idea: Brute Force

The most obvious approach is:

Step 1

Put every matrix element into an array.

```
1 3 5
2 6 9
3 6 9

↓
[1, 3, 5, 2, 6, 9, 3, 6, 9]
```

Step 2

Sort it.

```
[1, 2, 3, 3, 5, 6, 6, 9, 9]
```

Step 3

Return the middle element.

```
C++
return arr[(n * m) / 2];
```

Complexity

There are $n*m$ elements.

Sorting takes:

```
O(n*m log(n*m))
```

With $n, m \leq 400$, this can work, but the problem expects a more clever solution.

3. Can we use the fact that rows are sorted?

Yes!

Consider:

```
1 3 5
2 6 9
3 6 9
```

Suppose I ask:

How many elements are ≤ 5 ?

We can answer this **quickly in every row** because each row is sorted.

Row 1

```
1 3 5
```

All 3 elements are ≤ 5 .

```
count = 3
```

Row 2

```
2 6 9
```

Only 2 is ≤ 5 .

```
count = 1
```

Row 3

```
3 6 9
```

Only 3 is ≤ 5 .

```
count = 1
```

Total:

```
3 + 1 + 1 = 5
```

So there are **5 elements** ≤ 5 .

And since there are 9 elements total:

```
[1, 2, 3, 3, 5, 6, 6, 9, 9]
          ↑
          5
```

5 is the 5th element.

4. The key intuition

This is the important part.

Instead of trying to directly find the median, let's **guess an answer**.

Suppose we guess:

```
mid = 5
```

Then ask:

How many elements in the entire matrix are ≤ 5 ?

If the answer is at least the required position, then the median could be 5 or something smaller.

If the answer is too small, the median must be larger.

This creates a **monotonic condition**.

5. Why can we binary search?

Let's take the same matrix.

```
1 3 5
2 6 9
3 6 9
```

Imagine different values:

Candidate	Number of elements $\leq$ candidate
1	1
2	2
3	4
4	4
5	5
6	7
7	7
8	7
9	9

We need:

```
(n*m)/2 + 1
= 9/2 + 1
= 5
```

So we need the **smallest value whose count is at least 5**.

Look at the table:

```
candidate = 4 → count = 4 ❌
candidate = 5 → count = 5 ✅
```

Therefore:

```
answer = 5
```

This is exactly what **binary search on the answer** does.

6. What range do we binary search?

We need a minimum and maximum possible value.

Because every row is sorted:

```
C++
minimum = mat[i][0]
maximum = mat[i][m-1]
```

We can simply take:

```
C++
low = mat[0][0];
high = mat[0][m-1];
```

But this only works if the first row contains the global minimum and maximum, which isn't guaranteed.

So safer:

```
C++
low = INT_MAX;
high = INT_MIN;

for each row:
    low = min(low, mat[i][0]);
    high = max(high, mat[i][m-1]);
```

For:

```
1 3 5
2 6 9
3 6 9
```

we get:

```
low = 1
high = 9
```

7. How do we count elements $\leq$ mid efficiently?

This is where the **row-wise sorted property** becomes useful.

For a row:

```
2 6 9
```

Suppose:

```
mid = 7
```

We want:

```
number of elements <= 7
```

Since the row is sorted:

```
2 6 | 9
```

There are 2 elements ≤ 7 .

We can find this using:

```
C++
upper_bound(row.begin(), row.end(), mid)
```

upper_bound returns an iterator pointing to the **first element > mid**.

For:

```
2 6 9
```

and:

```
mid = 7
```

it points to:

```
9
```

Its position is:

```
2
```

Therefore:

```
C++
upper_bound(...) - row.begin()
```

gives:

```
2
```

which is exactly the number of elements ≤ 7 .

8. Complete algorithm

Step 1

Find the minimum and maximum possible values.

```
low = minimum element
high = maximum element
```

Step 2

Calculate how many elements we need on the left side.

```
C++
required = (n * m) / 2 + 1;
```

For 3×3 :

```
required = 5
```

Step 3

Binary search between **low** and **high**.

For every **mid**:

1. Count how many elements are $\leq$ **mid**.
2. If count is less than required:
 - median must be bigger
 - **low** = **mid** + 1
3. Otherwise:
 - **mid** can be the answer
 - search left
 - **high** = **mid**

At the end:

```
low == high
```

and that is the median.

9. C++ Code

```
C++
class Solution {
public:
    int median(vector<vector<int>> &mat) {
        int n = mat.size();
        int m = mat[0].size();

        // Find minimum and maximum possible values
        int low = INT_MAX;
        int high = INT_MIN;

        for (int i = 0; i < n; i++) {
            low = min(low, mat[i][0]);
            high = max(high, mat[i][m - 1]);
        }

        // We need this many elements to be <= median
        int required = (n * m) / 2 + 1;

        // Binary search on answer
        while (low < high) {
            int mid = low + (high - low) / 2;

            int count = 0;

            // Count elements <= mid
            for (int i = 0; i < n; i++) {
```

```

        count += upper_bound(
            mat[i].begin(),
            mat[i].end(),
            mid
        ) - mat[i].begin();
    }

    if (count < required) {
        // Too few elements <= mid
        // Median must be bigger
        low = mid + 1;
    }
    else {
        // Enough elements <= mid
        // Try to find a smaller answer
        high = mid;
    }
}

return low;
}
};

```

10. Understanding the most important line

This line:

```

C++
count += upper_bound(mat[i].begin(), mat[i].end(), mid)
        - mat[i].begin();

```

may look complicated.

Let's break it down.

Suppose:

```

row = [2, 6, 9]
mid = 7

```

Then:

```

C++
upper_bound(row.begin(), row.end(), 7)

```

returns iterator to:

```

9

```

Its index is:

```

2

```

So:

```

C++
upper_bound(...) - row.begin()

```

is:

```

2

```

Therefore:

```

2 elements <= 7

```

11. Complete dry run

Matrix:

```

1 3 5
2 6 9
3 6 9

```

There are:

```

n = 3
m = 3
total = 9

```

Required position:

```
required = 9 / 2 + 1
          = 4 + 1
          = 5
```

So we need the **smallest value having at least 5 elements $\leq$ it.**

Initial range

Minimum:

```
1
```

Maximum:

```
9
```

Therefore:

```
low = 1
high = 9
```

Iteration 1

```
low = 1
high = 9
```

Calculate:

```
mid = 1 + (9 - 1) / 2
     = 5
```

Now count elements ≤ 5 .

Row 1

```
1 3 5
```

Elements ≤ 5 :

```
1, 3, 5
```

count = 3

Row 2

```
2 6 9
```

Elements ≤ 5 :

```
2
```

count = 1

Row 3

```
3 6 9
```

Elements ≤ 5 :

```
3
```

count = 1

Total:

```
count = 3 + 1 + 1
       = 5
```

We need:

```
5
```

We have:

```
5
```

So `mid = 5` is a valid candidate.

Can there be a smaller answer?

Yes, let's search left:

```
high = 5
```

Now:

```
low = 1
high = 5
```

Iteration 2

```
mid = 1 + (5 - 1) / 2
    = 3
```

Count elements ≤ 3 .

Row 1

```
1 3 5
```

```
1, 3
```

count = 2

Row 2

```
2 6 9
```

```
2
```

count = 1

Row 3

```
3 6 9
```

```
3
```

count = 1

Total:

```
count = 2 + 1 + 1
      = 4
```

But required:

```
5
```

We don't have enough elements.

Therefore median must be **greater than 3**.

```
low = mid + 1
    = 4
```

Now:

```
low = 4
high = 5
```

Iteration 3

```
mid = 4 + (5 - 4) / 2
    = 4
```


Count elements ≤ 4 .

Row 1

```
1 3 5
```

2 elements:

```
1, 3
```

Row 2

```
2 6 9
```

1 element:

```
2
```

Row 3

```
3 6 9
```

1 element:

```
3
```

Total:

```
2 + 1 + 1 = 4
```

Still less than required:

```
4 < 5
```

Therefore:

```
low = 4 + 1
    = 5
```

Now:

```
low = 5
high = 5
```

Binary search stops.

Answer:

```
5
```

12. Why does `low` give the answer?

Our binary search is looking for:

The smallest value x such that at least $(n*m)/2 + 1$ elements are $\leq x$.

For our example:

```
x = 1 → 1 element ≤ x
x = 2 → 2
x = 3 → 4
x = 4 → 4
x = 5 → 5 ← first time we reach 5
```

Therefore:

```
answer = 5
```

This is a very common **Binary Search on Answer** pattern.

13. Complexity

Let:

$$N = n * m$$

Binary search on values

The values are at most **2000**.

Generally, binary search takes:

$$O(\log(\text{maxValue} - \text{minValue}))$$

Counting for one mid

For every row, we perform:

```
C++
upper_bound()
```

which takes:

$$O(\log m)$$

for each of **n** rows.

Therefore:

$$O(n \log m)$$

for one binary-search iteration.

Overall:

$$O(n \log m \log(\text{maxValue} - \text{minValue}))$$

With the given constraint `mat[i][j] <= 2000`, this is very fast.

Space

We don't create another array.

$$O(1)$$

extra space.

14. The intuition to remember

Don't memorize the code. Remember this chain:

```
Matrix is row-wise sorted
↓
For any value x, we can count
how many elements are <= x
↓
Use upper_bound() on every row
↓
We need (n*m)/2 + 1 elements <= median
↓
Find the smallest x satisfying this
↓
Binary search on x
↓
Answer = median
```

The one-line idea

Binary search the value of the median, and for each candidate use `upper_bound` in every sorted row to count how many elements are $\leq$ that candidate.

Exactly — **this is the most important conceptual point** of this problem.

You are asking:

"If I'm looking for an element at some index, why am I binary-searching over the **values** instead of the **indices**? And what happens if `mid` itself isn't present in the matrix?"

The answer is: **we are not binary-searching for an existing element directly. We are binary-searching for the smallest value that has enough elements smaller than or equal to it.**

Let's break that down.

1. We are NOT searching for an index

Normally, binary search looks like:

```
array = [1, 3, 5, 7, 9]
find 7
```

We search over indices:

```
0 1 2 3 4
```

because we're trying to find a particular existing element.

But here, we don't know where the median is.

We only know its **rank**.

For 3×3 :

```
total elements = 9
median position = 5th element
```

So our actual question becomes:

What value is the 5th smallest value?

That's different from:

"What is the element at index 4?"

2. Why search over values?

Consider:

```
1 3 5
2 6 9
3 6 9
```

After sorting:

```
1 2 3 3 5 6 6 9 9
```

We want the 5th element:

```
1 2 3 3 [5] 6 6 9 9
          ↑
```

Now imagine we don't know that 5 is the answer.

Instead, let's ask questions about **values**.

Ask:

Are there at least 5 elements ≤ 4 ?

No.

```
1 2 3 3
```

Only 4 elements.

So:

```
4 cannot be the median
```

Now ask:

Are there at least 5 elements ≤ 5 ?

Yes.

```
1 2 3 3 5
```

There are 5.

Therefore:

```
5 is a possible median
```

So we are effectively finding:

```
smallest value x
such that
count(elements <= x) >= required
```

That's why we binary-search **values**.

3. But what if `mid` isn't present?

This is the beautiful part.

Suppose our matrix was:

```
1 3 5
2 6 9
3 6 9
```

Our binary search might produce:

```
mid = 4
```

But **4 doesn't exist anywhere in the matrix!**

Is that a problem?

No.

Because 4 is only a **candidate threshold**.

We ask:

How many elements are `<= 4`?

They are:

```
1 2 3 3
```

Count:

```
4
```

We need:

```
5
```

Therefore:

```
count(<= 4) < 5
```

So we know:

```
median > 4
```

And we move:

```
C++
low = mid + 1;
```

which gives:

```
low = 5
```

And 5 actually exists.

4. Think of `mid` as a boundary, not an element

This is the key mental model.

When we do:

```
C++
```

```
mid = (low + high) / 2;
```

mid does NOT need to exist in the matrix.

We're asking:

```

      values in matrix
          ↓
1  2  3  3  5  6  6  9  9
-----|-----
          4

```

We're asking:

How many values are on the left side of 4?

Answer:

```
4
```

Then:

```
4 values <= 4
```

But we need 5.

Therefore, the median is somewhere to the right.

5. Why does `count <= mid` work?

Because this count is **monotonic**.

For example:

x	count(x <=)
1	1
2	2
3	4
4	4
5	5
6	7
7	7
8	7
9	9

Notice something:

```
1 → 2 → 4 → 4 → 5 → 7 → 7 → 7 → 9
```

The count **never decreases**.

This is exactly what binary search needs.

6. This also explains `upper_bound`

Suppose:

```
row = [2, 6, 9]
```

and:

```
mid = 7
```

7 doesn't exist.

But:

```
C++
upper_bound(row.begin(), row.end(), 7)
```

finds the first element **greater than 7**:

```
2 6 | 9
    ↑
```

Therefore, everything before it is `<= 7`.

```
2 6
```

So count = 2.

That's why `upper_bound` is perfect here.

7. Compare this with normal binary search

Normal binary search

You have:

```
[1, 3, 5, 7, 9]
```

and want to find:

```
7
```

You're searching for an **existing element**.

So you search indices:

```
low = 0
high = 4
```

This problem

We're trying to find:

```
5th smallest value
```

We don't know what that value is.

So we search the **value space**:

```
low = smallest matrix value
high = largest matrix value
```

Then for every candidate:

```
mid
```

we ask:

```
How many elements <= mid?
```

8. Another example where `mid` is NOT present

Consider:

```
4 8 12
5 9 13
6 10 14
```

Sorted:

```
4 5 6 8 9 10 12 13 14
```

Median:

```
9
```

Suppose binary search chooses:

```
mid = 10
```

10 exists, okay.

But suppose it chooses:

```
mid = 11
```

11 does **not** exist.

We calculate:

```
count(<= 11)
```

Values:

```
4 5 6 8 9 10
```

Count:

```
6
```

We need:

```
5
```

Therefore:

```
count >= required
```

So 11 is a valid **threshold**.

But we're looking for the **smallest valid threshold**, so we move left:

```
C++
high = 11;
```

Eventually we'll reach:

```
9
```

because:

```
count(<= 8) = 4 
count(<= 9) = 5 
```

Therefore:

```
answer = 9
```

9. The most important idea

Don't think:

"I'm binary-searching for the median element."

Think:

"**I'm binary-searching for a value boundary.**"

We're finding the smallest x for which:

```
number of elements <= x
>=
number of elements needed for median
```

Mathematically:

```
find minimum x such that
count(x) >= (n*m)/2 + 1
```

And because `count(x)` is monotonic:

```
false false false false true true true
                ↑
            first valid x
```

Binary search finds that first `true`.

10. Why the final `low` must be an actual matrix value

This is another subtle but important point.

Even though intermediate `mid` values can be absent, the **final answer will be an actual matrix value**.

Why?

Suppose the sorted elements are:

```
1 2 3 3 5 6 6 9 9
```

Consider the count function:

```
x = 4 → count = 4
x = 5 → count = 5
```

The count jumps from 4 to 5 exactly when we cross an actual element (5).

The **first value where the count reaches the required rank is therefore an actual element.**

That's why the binary search eventually lands on the median.

Remember this pattern

Whenever you see a problem asking:

"Find the kth smallest/largest value"

and you can efficiently answer:

"How many elements are $\leq x$?"

you should immediately think:

```

      kth smallest
        ↓
    binary search value
        ↓
    choose candidate x
        ↓
count elements ≤ x
        ↓
count < k ? move right
count ≥ k ? move left
        ↓
      answer
  
```

So yes, `mid` can absolutely be absent from the matrix. That's completely fine — `mid` is a value used to divide the search space, not necessarily an element we're trying to find.

ChatGPT can make mistakes. Check important info.